CMSC131: Recursion and Recursive Tracing

Chapter 11

By Naren Ayinala

May 2026

Contents

1 The Big Picture: A Method That Calls Itself **3**

2 Our First Example: A Countdown **3**

2.1 The Call Chain Is Not One Variable Changing 3

2.2 What If We Forget the Base Case? 4

3 How Java Executes Recursive Calls: The Call Stack **4**

4 Tracing Recursion Without Guessing **5**

5 Factorial: A Classic Returning Example **6**

6 Printing Before Versus After the Recursive Call **6**

6.1 A Mixed Trace 6

7 Recursion Versus Iteration **7**

8 Recursion with Strings **7**

8.1 Printing a String Backward 7

8.2 Counting a Character 7

9 Recursion with Arrays: Use an Index **8**

9.1 Find a Value Recursively 8

10 Helper Methods and Public Interfaces **8**

11 Recursion with More Than One Recursive Call **9**

12 A Recursive Pattern with Two Indices: Palindromes **9**

13 Writing Recursive Methods Step by Step **10**

14 Common Mistakes and How to Debug Them **10**

14.1 Missing or Unreachable Base Case 10

14.2 Calling the Method Without Using Its Return Value 11

14.3 Confusing a Return with Printing 11

14.4 Tracing Tips 11

15 Worked Tracing Exercises **11**

15.1 Exercise A: The Order of Prints 11

15.2 Exercise B: A Returning Method 12

15.3 Exercise C: Spot the Bug 12

16 Chapter Summary **12**

1 The Big Picture: A Method That Calls Itself

So far, our Java programs have used loops to repeat work. A **recursive method** repeats work differently: it calls *itself* to solve a smaller version of the same problem. Recursion is not a special Java keyword or a different kind of method. It is an ordinary method call where the caller and the method being called have the same name.

Imagine a stack of five books. To count the books, you could count them one at a time. Or you could say: "There is one book on top, plus however many books are in the smaller stack underneath." Continue until the stack is empty. That is the recursive idea: **reduce a large problem to a smaller problem of the same kind**.

Every correct beginner-level recursive solution needs two pieces:

1. A **base case**: the simplest input, answered immediately without another recursive call.
2. A **recursive case**: an action that calls the method on a smaller or simpler input that moves toward the base case.

> Recursion is not automatically better than a loop. It is another way to express repetition. We use it when reducing a problem into smaller copies makes the solution easier to understand.

2 Our First Example: A Countdown

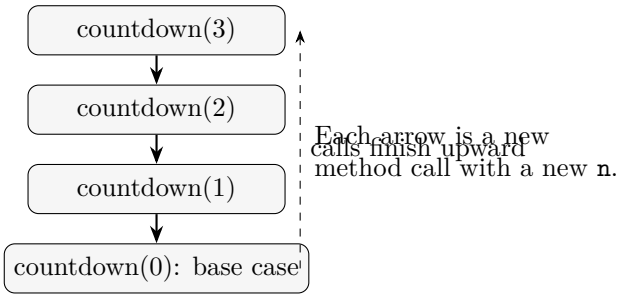
```
public static void countdown(int n) {
    if (n == 0) {
        System.out.println("Done!");
        return;
    }
    System.out.println(n);
    countdown(n - 1);
}

// Calling countdown(3) prints:
// 3
// 2
// 1
// Done!
```

Read the method from top to bottom. When `n` is zero, it prints `Done!` and returns. Otherwise, it prints the current number and calls `countdown(n - 1)`. The argument becomes smaller each time: 3, then 2, then 1, then 0.

2.1 The Call Chain Is Not One Variable Changing

Each call is a *new invocation* with its own parameter `n`. The earlier calls have not vanished; they are waiting for their inner calls to finish.



2.2 What If We Forget the Base Case?

```
public static void broken(int n) {
    System.out.println(n);
    broken(n - 1); // No stopping condition!
}
```

The method keeps creating more calls until the call stack runs out of space, normally resulting in a `StackOverflowError`. A base case is necessary, but not sufficient: the recursive call must also *reach* it. For example, a method that stops at `n == 0` but calls itself with `n + 1` starting from 3 will never reach zero.

3 How Java Executes Recursive Calls: The Call Stack

Recall that local variables and parameters belong to an individual method invocation. Java tracks active method calls using a **call stack**. When a method calls another method, a new stack frame is created. When that call returns, its frame is removed and execution resumes in the caller.

```
public static void show(int n) {
    if (n == 0) return;
    System.out.println("Enter " + n);
    show(n - 1);
    System.out.println("Leave " + n);
}

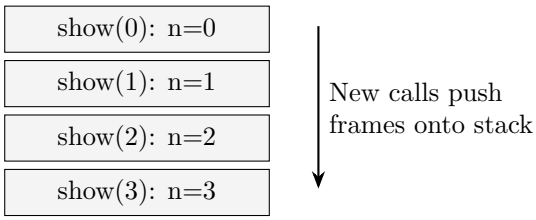
show(3);
```

Output:

```
Enter 3
Enter 2
Enter 1
Leave 1
Leave 2
Leave 3
```

Why does `Leave 3` print last? The first invocation pauses at `show(2)`. It cannot execute its next line until `show(2)` returns; that call itself waits for `show(1)`.

Stack while `show(0)` is active



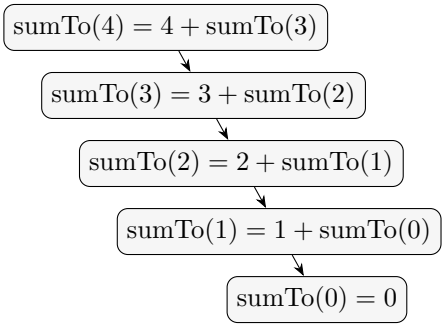
Bottom: oldest call; top: newest call

The *heap* is different: objects and arrays created with `new` live there. Recursive calls create stack frames, not automatic copies of every heap object they reference. If each recursive call receives the same array reference, all calls can access the same array.

4 Tracing Recursion Without Guessing

A useful strategy is to write down every call in order, identify the base case, then work *backward* as calls return. Consider this method:

```
public static int sumTo(int n) {
    if (n == 0) return 0;
    return n + sumTo(n - 1);
}
System.out.println(sumTo(4)); // 10
```



On the way down: create calls.
On the way back up: compute $1 + 0 = 1$, then $2 + 1 = 3$, then $3 + 3 = 6$, then $4 + 6 = 10$.

Invocation	Waits for	Eventually returns
`sumTo(0)`	nothing	0
`sumTo(1)`	`sumTo(0)`	$1 + 0 = 1$
`sumTo(2)`	`sumTo(1)`	$2 + 1 = 3$
`sumTo(3)`	`sumTo(2)`	$3 + 3 = 6$
`sumTo(4)`	`sumTo(3)`	$4 + 6 = 10$

When tracing `return n + recursiveCall(...)`, do *not* add `n` immediately. Java must evaluate the recursive call first. The caller is waiting for a value.

5 Factorial: A Classic Returning Example

Factorial is defined by $n! = n(n-1)(n-2)\cdots 1$, and $0! = 1$. The definition suggests the smaller subproblem $n! = n \cdot (n-1)!$.

```
public static int factorial(int n) {
    if (n < 0) {
        throw new IllegalArgumentException("n must be >= 0");
    }
    if (n == 0) return 1;
    return n * factorial(n - 1);
}

System.out.println(factorial(4)); // 24
```

Expansion: `factorial(4)` needs $4 \times$ `factorial(3)`, which needs $3 \times$ `factorial(2)`, and so on. At the base case, `factorial(0)` returns 1. Unwinding gives $1, 1, 2, 6, 24$ for calls with arguments 0 through 4.

The negative-input check is important. If we only check `n == 0` and call `factorial(-2)`, the argument moves farther away from zero. Also, an `int` cannot hold arbitrarily large factorials: integer overflow is a separate limitation from recursion.

6 Printing Before Versus After the Recursive Call

Moving a single statement can completely change the output.

```
public static void forward(int n) {
    if (n == 0) return;
    System.out.print(n + " ");
    forward(n - 1);
}

public static void backward(int n) {
    if (n == 0) return;
    backward(n - 1);
    System.out.print(n + " ");
}

forward(4); // 4 3 2 1
backward(4); // 1 2 3 4
```

In `forward`, printing happens on the **way down**. In `backward`, printing happens on the **way back up**, after the deeper calls return. The method invocations happen in the same argument order; the location of the print statement changes when the output is produced.

6.1 A Mixed Trace

```
public static void mystery(int n) {
    if (n <= 0) {
        System.out.print("X ");
        return;
    }
    System.out.print("A" + n + " ");
```

```
    mystery(n - 1);
    System.out.print("B" + n + " ");
}
// mystery(2): A2 A1 X B1 B2
```

To solve these questions on paper, draw an arrow down for each call and another arrow up for each return. Put "before" prints next to downward arrows and "after" prints next to upward arrows.

7 Recursion Versus Iteration

The sum of the first `n` nonnegative integers can be implemented either way:

```
public static int sumLoop(int n) {
    int total = 0;
    for (int i = 1; i <= n; i++) {
        total += i;
    }
    return total;
}

public static int sumRecursive(int n) {
    if (n == 0) return 0;
    return n + sumRecursive(n - 1);
}
```

Both calculate the same answer for nonnegative `n`. The loop updates variables within one invocation. The recursive method creates a chain of invocations, each with its own parameter and pending return expression. In introductory problems, use whichever representation makes the steps clearer; do not assume recursion is always the preferred approach.

8 Recursion with Strings

8.1 Printing a String Backward

A string can be reduced by removing its first character. This gives a natural recursive example.

```
public static void reversePrint(String text) {
    if (text.length() == 0) return;
    reversePrint(text.substring(1));
    System.out.print(text.charAt(0));
}
reversePrint("cat"); // tac
```

The calls are `"cat"`, `"at"`, `"t"`, and `""`. The empty string is the base case. The first characters print on the way back: `t`, then `a`, then `c`. Remember that `substring(1)` creates a new string value; Java strings are immutable.

8.2 Counting a Character

```
public static int countChar(String s, char target) {
```

```
    if (s.isEmpty()) return 0;
    int first = (s.charAt(0) == target) ? 1 : 0;
    return first + countChar(s.substring(1), target);
}
System.out.println(countChar("banana", 'a')); // 3
```

This method handles one character and asks the same question about the rest of the string. The result is assembled while calls return.

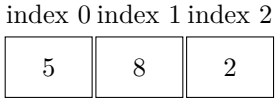
9 Recursion with Arrays: Use an Index

For arrays, we usually avoid copying smaller arrays. Instead, pass the *same* array along with an index describing how much has been processed.

```
public static int arraySum(int[] values, int index) {
    if (index == values.length) return 0;
    return values[index] + arraySum(values, index + 1);
}

int[] nums = {5, 8, 2};
System.out.println(arraySum(nums, 0)); // 15
```

The index starts at zero and increases until it equals `values.length`. That equality is the stopping condition. At the stopping point, do not access `values[index]`: that index is already outside the array.



Calls use indices
$0 \to 1 \to 2 \to 3$
Base case: index $= 3$

9.1 Find a Value Recursively

```
public static boolean contains(int[] a, int i, int target) {
    if (i == a.length) return false;
    if (a[i] == target) return true;
    return contains(a, i + 1, target);
}
```

There are two ways to stop: find the target or reach the end. This is still a correct recursive solution because every call either returns immediately or advances the index.

10 Helper Methods and Public Interfaces

Students should not always have to supply a starting index. A **helper method** lets us hide that bookkeeping from callers.

```
public static int sum(int[] values) {
    return sumHelper(values, 0);
}
```

```
private static int sumHelper(int[] values, int index) {
    if (index == values.length) return 0;
    return values[index] + sumHelper(values, index + 1);
}

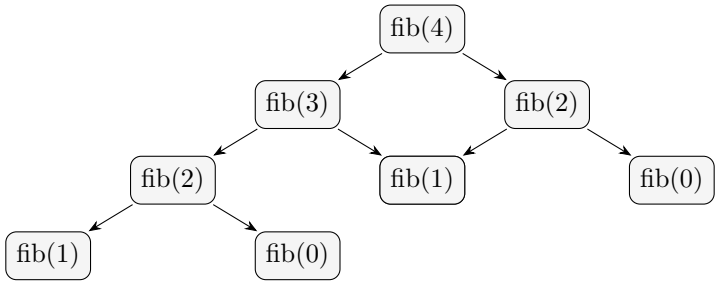
System.out.println(sum(new int[]{3, 4, 5})); // 12
```

The public method is convenient; the private recursive helper carries the changing index. This pattern also makes it easier to validate input before recursion starts.

11 Recursion with More Than One Recursive Call

Some problems branch into multiple smaller subproblems. Consider the Fibonacci sequence: $F(0) = 0$, $F(1) = 1$, and $F(n) = F(n-1) + F(n-2)$ for $n \geq 2$.

```
public static int fib(int n) {
    if (n < 0) throw new IllegalArgumentException();
    if (n <= 1) return n;
    return fib(n - 1) + fib(n - 2);
}
System.out.println(fib(5)); // 5
```



Notice that `fib(2)` is computed more than once. The tree illustrates that recursive calls can repeat work. At this stage, the important lesson is how to trace the call tree correctly, not to memorize algorithmic complexity. A loop is often simpler for calculating Fibonacci numbers in practice.

12 A Recursive Pattern with Two Indices: Palindromes

A palindrome reads the same forward and backward. Compare the first and last characters; if they match, solve the smaller problem inside them.

```
public static boolean isPalindrome(String s) {
    return palindromeHelper(s, 0, s.length() - 1);
}

private static boolean palindromeHelper(String s,
                                        int left, int right) {
    if (left >= right) return true;
    if (s.charAt(left) != s.charAt(right)) return false;
    return palindromeHelper(s, left + 1, right - 1);
}
```

```
System.out.println(isPalindrome("racecar")); // true
System.out.println(isPalindrome("java")); // false
```

Two indices move toward one another. The base case `left >= right` means no unmatched characters remain. The method also returns early if the current outer pair differs. This implementation is case-sensitive and treats spaces and punctuation as ordinary characters.

13 Writing Recursive Methods Step by Step

When faced with a new problem, use the following reasoning process rather than immediately writing a self-call.

1. **State what the method means.** For example, `digitSum(n)` returns the sum of the digits of a nonnegative integer.
2. **Find the smallest input.** For a single-digit nonnegative number, the answer is the number itself.
3. **Identify a smaller version.** Remove the last digit with integer division by 10.
4. **Combine one piece with the smaller answer.** The last digit is `n % 10`.
5. **Check progress.** For a number with two or more digits, `n / 10` is smaller and eventually becomes a single digit.

```
public static int digitSum(int n) {
    if (n < 0) throw new IllegalArgumentException();
    if (n < 10) return n;
    return (n % 10) + digitSum(n / 10);
}
System.out.println(digitSum(472)); // 13
```

Tracing `digitSum(472)`: $2+$`digitSum(47)`, then $7+$`digitSum(4)`, then 4. On the way back: $7+4=11$, then $2+11=13$.

14 Common Mistakes and How to Debug Them

14.1 Missing or Unreachable Base Case

```
// Wrong for n = 3: n moves away from zero.
public static void wrong(int n) {
    if (n == 0) return;
    wrong(n + 1);
}
```

Ask: "What values will the argument take?" If the sequence never reaches a base case, the recursion does not terminate normally.

14.2 Calling the Method Without Using Its Return Value

```
// Incorrect: the result of the recursive call is discarded.
public static int badSum(int n) {
    if (n == 0) return 0;
    badSum(n - 1);
    return n;
}

// Correct:
public static int goodSum(int n) {
    if (n == 0) return 0;
    return n + goodSum(n - 1);
}
```

A recursive call is still an ordinary method call. If it returns an `int`, you must decide whether to use that returned value. Calling it and ignoring its result is usually not what a sum or factorial method needs.

14.3 Confusing a Return with Printing

`System.out.println(x)` displays a value but does not return it to the caller. `return x` gives a value back to the waiting method invocation. The distinction matters especially when recursive calls need to combine answers.

14.4 Tracing Tips

When debugging, write the argument at entry and the return value at exit. For a returning method, use a small table with columns "call," "base case?," "waiting for," and "returns." If a method prints before and after recursion, mark both locations separately. A debugger's *step into* command can show new stack frames being created.

15 Worked Tracing Exercises

Try to determine each output before reading the explanation.

15.1 Exercise A: The Order of Prints

```
public static void trace(int n) {
    if (n == 0) {
        System.out.print("Z ");
        return;
    }
    System.out.print(n + " ");
    trace(n - 1);
    System.out.print(n * 10 + " ");
}
trace(3);
```

Answer: `3 2 1 Z 10 20 30`. The values 3, 2, 1 print as calls descend; Z prints at the base case; 10, 20, 30 print as calls return.

15.2 Exercise B: A Returning Method

```
public static int mysteryNumber(int n) {
    if (n <= 1) return 1;
    return 2 + mysteryNumber(n - 2);
}
System.out.println(mysteryNumber(6));
```

Answer: 7. The calls are 6, 4, 2, 0. The base returns 1; the other three calls each add 2: $1+2+2+2=7$.

15.3 Exercise C: Spot the Bug

```
public static int countDownSum(int n) {
    if (n == 1) return 1;
    return n + countDownSum(n - 1);
}
```

For positive `n`, this works. But `countDownSum(0)` never reaches 1, because it keeps calling with negative arguments. Fix it by validating `n >= 0` and using `if (n == 0) return 0;` as the base case.

16 Chapter Summary

Concept	What to remember
Recursive method	A method that calls itself on a smaller subproblem.
Base case	A case that returns without making another recursive call.
Recursive case	A call that makes measurable progress toward a base case.
Call stack	Each invocation has its own parameters and locals; callers wait for callees.
Unwinding	After the base case returns, waiting expressions finish in reverse call order.
Before/after print	Before recursion prints while descending; after prints while returning.
Array recursion	Pass the same array with a changing index, often using a helper.
Multiple calls	One invocation may branch into several recursive subproblems.
Stack overflow	Often caused by missing stopping logic or excessively deep calls.

The most important recursion habit: identify the base case, write the exact sequence of arguments, and then trace return values from the deepest call back to the original caller.